

软件过程与管理 课后作业 Assgt_01

姓名：李秉琰学号：2023141450139 日期：2025 年 4 月

一、项目背景简介

本次选取本人在大三期间参与开发的"校园二手物品交易平台"作为对象，对该项目的软件过程进行定义与描述。该项目由 4 人小组完成，采用前后端分离架构，前端使用 Vue.js，后端使用 Spring Boot，数据库为 MySQL，开发周期约为 12 周。

二、软件过程定义

依据 ISO/IEC 12207:2017 对软件生存周期过程的框架划分，结合项目实际规模与团队情况，本项目的软件过程定义如下：

2.1 协议过程（Agreement Processes）

本项目为课程设计作业，无正式甲乙双方合同，协议过程以小组内部需求确认代替，通过需求评审会议形成书面需求说明，作为后续开发的基线。

2.2 项目使能过程（Project Enabling Processes）

过程名称	具体活动
------	------

项目规划	制定项目计划书，明确里程碑与任务分工
------	--------------------

项目评估与控制	每周例会同步进度，使用 GitHub Issues 追踪问题
---------	--------------------------------

配置管理	使用 Git 进行版本控制，主干分支保护，功能分支开发
------	-----------------------------

质量保证	代码合并前进行 Code Review，关键功能编写单元测试
------	--------------------------------

2.3 技术过程（Technical Processes）

（1）需求定义过程

- 通过访谈与问卷收集潜在用户需求
- 编写用户故事（User Story），整理形成功能需求列表
- 使用 MoSCoW 方法对需求优先级排序（Must / Should / Could / Won't)
- 输出物：需求规格说明书（SRS）

(2) 系统/软件架构设计过程

- 确定整体系统架构：前后端分离，RESTful API 通信
- 划分模块：用户管理、商品发布、订单管理、即时消息、搜索推荐
- 数据库 ER 图设计，接口文档编写（使用 Swagger）
- 输出物：概要设计说明书、数据库设计文档、接口文档

(3) 实现过程

- 按功能模块并行开发，遵循统一编码规范
- 前端组件化开发，后端 Controller-Service-Mapper 三层架构
- 每个迭代周期为 2 周，共 3 个迭代
- 输出物：源代码、单元测试用例

(4) 集成过程

- 各模块完成后进行接口联调测试
- 使用 Postman 进行接口回归测试
- 持续集成：通过 GitHub Actions 自动化构建与测试
- 输出物：集成测试报告

(5) 验证与确认过程

- 验证 (Verification)：对照需求规格说明书，逐条确认功能实现是否符合设计
- 确认 (Validation)：邀请 5 名同学进行用户验收测试 (UAT)，收集反馈
- 输出物：测试报告、用户验收记录

(6) 部署过程

- 将系统部署至云服务器（阿里云 ECS），配置 Nginx 反向代理
- 编写部署说明文档，记录环境配置步骤
- 输出物：部署说明文档

(7) 运维支持过程

- 项目周期内提供基本维护：修复测试反馈的 Bug
- 记录问题日志，版本迭代修复
- 输出物：问题追踪记录 (GitHub Issues)

三、软件过程定义的来源

本次软件过程的定义主要来源于以下三个标准，并结合项目实际进行综合参考：

(1) ISO/IEC 12207:2017（软件生存周期过程）

ISO/IEC 12207 提供了完整的软件生存周期过程框架，将过程划分为协议过程、项目使能过程、技术过程和组织使能过程四大类。本项目的整体过程结构以该标准为主要参考，按照其过程分类方式组织各阶段活动，确保过程覆盖的完整性与规范性。

(2) ISO/IEC 15504（软件过程评估，即 SPICE）

ISO/IEC 15504 提供了软件过程能力评估的维度，定义了从“不完整”到“优化”共 6 个能力等级。参考该标准，本项目在各过程中明确定义了**输入、活动、输出**三要素，确保过程可被执行、可被评估，对照能力等级目标达到“已管理级（Level 2）”的基本要求。

(3) ISO/IEC 33001（过程评估概念与术语）

ISO/IEC 33001 作为 15504 系列的概念基础，明确了“过程”、“过程属性”、“过程能力”等核心术语的定义。本项目在描述软件过程时，遵循该标准对过程要素的定义方式，保证术语使用的准确性与一致性。

四、过程裁剪的想法

软件过程裁剪（Tailoring）是指根据项目的具体特征，对标准过程框架进行适当的简化、调整或增强，以在质量保证与开发效率之间取得平衡。

本项目的裁剪依据和思路如下：

(1) 裁剪依据

裁剪因素	本项目情况
项目规模	小型项目，4 人团队，12 周开发周期
合同/协议要求	课程作业，无正式合同约束
风险程度	低风险，不涉及安全关键系统
组织成熟度	学生团队，过程能力相对有限

(2) 裁剪内容

- **简化协议过程：**省略正式的采购合同与供应商管理，以团队内部需求确认文档

代替

- **简化质量保证过程**：未设立独立的 QA 角色，由全体成员共同承担质量职责，采用轻量级 Code Review 替代正式审查
- **合并验证与确认过程**：由于团队规模小，将 V&V 合并为一个测试阶段统一执行
- **省略独立的度量与分析过程**：以每周例会的进度汇报代替正式的度量数据收集与分析
- **保留并强化配置管理过程**：Git 分支管理与 GitHub Actions CI 是保证多人协作质量的核心，予以保留并强化

(3) 裁剪结论

通过上述裁剪，在保留技术过程核心活动（需求→设计→实现→测试→部署）完整性的前提下，去除了不适用于小型学生项目的管理开销，使过程执行更加轻量、高效，符合项目的实际规模与能力约束。

参考标准：ISO/IEC 12207:2017, ISO/IEC 15504, ISO/IEC 33001
